

МИНОБРНАУКИ РОССИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ

ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт Компьютерных наук и технологий

Высшая школа искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Программирование и алгоритмизация
Курсовая работа "Игра Уно"

Студент: _____ Луговенко П.В.

Преподаватель: _____ Сеннов В. Н.

« _____ » _____ 2023г.

Санкт-Петербург, 2023

Аннотация

Курсовая работа представляет из себя реализацию класса игрока в "Уно" на высокоуровневом языке программирования C++ с использованием объектно-ориентированных принципов. Работа выполнена в среде разработки Microsoft Visual Studio 2022 версии 17.5.2.

Содержание

1	Постановка задачи	4
2	Используемые абстракции	5
2.1	Игрок	5
2.2	Карта	5
2.3	Рука	5
3	Математическое описание алгоритмов	6
3.1	Poly_player	6
3.1.1	Получение карт	6
3.1.2	Смена цвета	6
3.1.3	Получение информации о количестве карт игроков	6
3.1.4	Ход активного игрока	6
3.1.5	Получение дополнительной карты	6
3.2	Luser_player	7
3.2.1	Игрок получает карты	7
3.2.2	Смена цвета	7
3.2.3	Ход активного игрока	7
3.2.4	Получение дополнительной карты	7
4	Реализация	8
4.1	Абстракции	8
4.2	Алгоритмы	8
4.2.1	Получение карт	8
4.2.2	Смена цвета	9
4.2.3	Получение информации о количестве карт игроков	10
4.2.4	Ход активного игрока	11
4.2.5	Получение дополнительной карты	11
5	Результаты	13
	Заключение	14
	Приложение А. Исходный код Luser_player	16
A.1	Luser_player.h	16
A.2	Luser_player.cpp	17
	Приложение Б. Исходный код Poly_player	20
Б.1	Poly_player.h	20
Б.2	Poly_player.cpp	22

1 Постановка задачи

Для выполнения курсовой работы выдается исходный код[1] программы, реализующей игру в "Уно".

Цель работы: реализация класса игрока в "Уно", с которым корректно работает выданный код.

Для достижения цели работы необходимо было выполнить следующие задачи:

- 1) унаследовать класс от абстрактного класса игрока, с которым работает выданный код;
- 2) реализовать класс успешного игрока посредством внедрения алгоритмов, улучшающих стратегию;
- 3) написать отчёт по написанному модулю.

2 Используемые абстракции

2.1 Игрок

Игрок - это абстракция, способная взаимодействовать с игрой, наблюдать за событиями игры и реагировать на них. Игрок обладает некоторыми стратегиями, которые помогают ему одержать победу над другими игроками. Чтобы игра проходила корректно и реалистично, сущность игрока должна обладать такими характеристиками, как имя и рука с картами, а так же игрок должен понимать, при каком количестве карт соперник близится к победе.

2.2 Карта

Карта - это абстракция, которая обладает такими характеристиками, как цвет(синий, жёлтый, красный, зелёный) и значение. В зависимости от значения карты, она относится либо к обычной карте (с номером 0 до 9), либо к карте действия(«Возьми две» («Draw 2»), «Ход обратно» («Reverse»), «Пропусти ход» («Skip»), «Заказать цвет» («Wild Card»), («Wild Draw 4 Card») .

2.3 Рука

Рука - множество карт, которое имеет игрок в данный момент игры.

3 Математическое описание алгоритмов

В данном разделе изложено математическое описание алгоритмов реализованных игроков.

3.1 Poly_player

Poly_player - это сильный игрок, обладающий некоторыми стратегиями, которые позволяют одержать победу над слабым игроком. Ниже представлено описание алгоритмом, реализующих успешную стратегию игрока.

3.1.1 Получение карт

Игрок получает карты на руки. В зависимости от момента получения карт (начало новой партии/использование карты действия) «рука» последовательно заполняется определенным количеством карт.

3.1.2 Смена цвета

Подсчитываем количество карт каждого цвета в руке игрока, далее выбираем этот цвет и сбрасываем соответствующую карту. Цвет сброшенной карты становится текущим.

3.1.3 Получение информации о количестве карт игроков

Смотрим на направление игры, чтобы правильно определить соседнего игрока. Далее получаем информацию о количестве карт соседа.

3.1.4 Ход активного игрока

Получаем информацию о количестве карт соседнего игрока. Заданное количество карт, означающее приближение игрока к победе, известно заранее. Обозначим его переменной n . Если количество карт меньше n , то есть соседний игрок близится к победе, активный игрок использует карты действия, если такие имеются в руке игрока. Очевидно, что любая из карт действия лишает соседнего игрока хода, и тем самым уменьшает шанс его победы. К тому же, карты действия тяжелее обычных, а значит, их не выгодно копить. В случае отсутствия карт действия, активный игрок по возможности меняет именно цвет карты сброса. Если у соседнего игрока много карт, то активный игрок первым делом старается поменять цвет карты сброса, то есть скинуть карту, схожую по значению. Если таких карт нет, то игрок скидывает самую «тяжелую» (с самым большим значением) карту схожего цвета. В оставшемся случае игрок выкидывает дикую карту, больший приоритет имеет карта «Wild Draw 4 Card».

3.1.5 Получение дополнительной карты

Берется карта из колоды. Далее сравниваются цвет и значения взятой карты с картой сброса. Если совпадение нашлось, то карта отправляется в сброс, иначе игрок забирает карту себе в руку.

3.2 Luser_player

Luser_player - это слабый игрок, который соблюдает правила игры, но не использует никаких стратегий. Ниже представлено описание алгоритмов, реализующих корректную работу игрока.

3.2.1 Игрок получает карты

Игрок получает карты на руки. В зависимости от момента получения карт (начало новой партии/использование карты действия) «рука» последовательно заполняется определенным количеством карт.

3.2.2 Смена цвета

Если игроку необходимо сменить цвет, то он меняет его на рандомный.

3.2.3 Ход активного игрока

Сравниваются карта сброса и карты в руке игрока. При первом совпадении по цвету или числовому значению игрок скидывает карту.

3.2.4 Получение дополнительной карты

Берется карта из колоды. Далее сравниваются цвет и значения взятой карты с картой сброса. Если совпадение нашлось, то карта отправляется в сброс, иначе игрок забирает карту себе в руку.

4 Реализация

4.1 Абстракции

Абстракция игрок - это производный класс от абстрактного класса UnoPlayer, который используется классом игры UnoGame. В ходе выполнения курсовой работы класс UnoPlayer был расширен некоторыми функциями, которые улучшают стратегию игрока и проверяют соблюдение правил игры. Также в класс «игрока» была добавлена дополнительная характеристика для реализации некоторых методов. Всего класс игрока Poly_player имеет три поля, класс игрока Luser_player имеет два поля(без последнего).

- 1) Player_name - поле класса, отвечающее за хранение имени «игрока»;
- 2) hand_card - поле класса, отвечающее за хранение «карт» в «руке» у «игрока»;
- 3) cards - поле класса, отвечающее за контроль количества карт у «соседнего игрока». В разделе 3.1.4 данное понятие обозначается переменной n.

Карта - это экземпляр структуры Card. «Карта» располагает такими полями, как цвет - color и значение - value.

Рука реализована в виде массива указателей на карты - структуру Card.

4.2 Алгоритмы

4.2.1 Получение карт

Алгоритм получения карт реализуется при помощи метода «receiveCards», который является производным методом от абстрактного класса UnoPlayer. Основа метода - цикл типа «for-each» по массиву карт, который хранится в руке. В цикле происходит заполнение массива нужным количеством карт. Код для игрока Poly_player представлен в [листинге 1](#). Код для игрока Luser_player представлен в [листинге 2](#).

Листинг 1.

```
1  /// @brief Игрок получает на руки карты.
2  /// @param cards массив карт.
3  void Poly_player::receiveCards(const vector<const Card*>& cards)
4  {
5      if (cards.size() == 7)
6      {
7          hand_card.clear();
8      }
9      for (const Card* cur_card : cards)
10     {
11         this->hand_card.push_back(cur_card);
12     }
13 }
```

Листинг 2.

```
1  void Luser_player::receiveCards(const vector<const Card*>& cards)
2  {
```



```

3   if (cards.size() == 7)
4   {
5       hand_card.clear();
6   }
7   for (const Card* cur_card : cards)
8   {
9       this->hand_card.push_back(cur_card);
10  }
11 }

```

4.2.2 Смена цвета

Алгоритм смены цвета реализуется при помощи метода «changeColor», который является производным методом от абстрактного класса UnoPlayer. Метод, реализованный в классе игрока Poly_player, содержит цикл типа «for-each» по массиву карт, который хранится в руке. В цикле при помощи конструкции «switch-case» происходит подсчет карт определенного цвета. Таким образом, функция возвращает самый часто встречающийся цвет в руке игрока. Код для игрока Poly_player представлен в [листинге 3](#). Код для игрока Luser_player представлен в [листинге 4](#).

Листинг 3.

```

1  /// @brief Если игрок положил "Закажи цвет" или "Возьми четыре", то
2  /// игра запросит у него новый цвет. функция возвращает самый
3  /// часто встречающийся цвет в руке игрока.
4  /// @return новый цвет.
5  CardColor Poly_player::changeColor()
6  {
7      int num_color[4] = {0};
8      for (const Card* cur_card : hand_card)
9      {
10         switch (cur_card->color)
11         {
12             case 0:
13                 num_color[0] += 1;
14                 break;
15             case 1:
16                 num_color[1] += 1;
17                 break;
18             case 2:
19                 num_color[2] += 1;
20                 break;
21             case 3:
22                 num_color[3] += 1;
23                 break;
24         }
25     }
26     int index = 0;
27     int max = 0;
28     for (int i = 0; i < 4; i++)
29     {
30         if (num_color[i] > max)
31         {
32             max = num_color[i];
33             index = i;

```

```

34     }
35 }
36 return CardColor(index);
37 }

```

Листинг 4.

```

1 CardColor Luser_player::changeColor()
2 {
3     srand(time(NULL));
4     int max = 3;
5     int num = rand() % (max + 1);
6
7     return CardColor(num);
8
9 }

```

4.2.3 Получение информации о количестве карт игроков

Алгоритм получения информации о количестве карт реализуется методами «NumberCardNeighboringPlayer» и «ToxicFunction». В методе первым делом происходит получение индекса активного игрока и массива карт всех игроков. Далее с учетом направления игры рассчитывается индекс соседнего игрока. Рассматривается три ситуации:

- 1) выход за границы с начала массива. В этом случае индекс соседнего игрока равен индексу последнего игрока.
- 2) выход за границы с конца массива. В этом случае индекс соседнего игрока равен индексу первого игрока;
- 3) высчитанный индекс не выходит за границы массива.

С учётом рассмотренных ситуаций, получаем количество карт следующего игрока. Далее сравниваем его с заданным количеством карт и возвращаем true, если сосед близится к победе. Код для игрока Poly_player представлен в [листинге 3](#)

Листинг 5.

```

1 int Poly_player::NumberCardNeighboringPlayer()
2 {
3     int cards = 0;
4     int index = this->game()->activePlayerIndex();
5     std::vector<int> playersCards= game()->numberOfCards();
6     int offset = game()->currentDirection() == 0 ? 1 : -1;
7     if ((index + offset) == -1)
8     {
9         cards = playersCards[playersCards.size()-1];
10    }
11    else if ((index + offset) == playersCards.size())
12    {
13        cards = *playersCards.begin();
14    }
15    else
16    {

```

```

17     cards = playersCards[index+offset];
18 }
19     return cards;
20 }
21
22 bool Poly_player::ToxicFunction()
23 {
24     int score = NumberCardNeighboringPlayer();
25     if (score <= cards)
26     {
27         return true;
28     }
29     return false;
30 }

```

4.2.4 Ход активного игрока

Алгоритм хода активного игрока реализуется при помощи метода «playCard», который является производным методом от абстрактного класса UnoPlayer, а так же при помощи методов «PlayerHasFewCards», «PlayerHasNoWildCards», «PlayerHasWildCards». В последних трех методах рассматриваются три ситуации, в которой может оказаться активный игрок:

- 1) метод «PlayerHasFewCards» вызывается в случае, когда соседний игрок близится к победе;
- 2) метод «PlayerHasNoWildCards» вызывается в случае обычного хода игрока в течении игры;
- 3) метод «PlayerHasWildCards» вызывается в случае, когда ни одна из приведенных выше ситуаций не произошла;

Во всех описанных методах используется цикл типа «for-each» по массиву карт в руке игрока. Каждый из методов передает наиболее выгодную карту в метод «playCard», который возвращает карту хода игрока. После хода карта удаляется из руки при помощи метода «Reset». Реализация методов «playCard», «PlayerHasFewCards», «PlayerHasNoWildCards», «PlayerHasWildCards» и «Reset» для игрока Poly_player приведена в [приложении Б](#). Реализация методов «playCard», «PlayerHasNoWildCards», «PlayerHasWildCards» и «Reset» приведена в [приложении А](#).

4.2.5 Получение дополнительной карты

Алгоритм получения дополнительной карты реализуется при помощи метода «drawAdditionalCard», который является производным методом от абстрактного класса UnoPlayer. Метод сравнивает дополнительную карту с картой сброса по цвету или значению и в случае совпадения возвращает true - игрок скинул дополнительную карту. Иначе функция возвращает false, а карта добавляется в руку игрока при помощи функции Vector Push_Back(). Код для игроков Poly_player и Luser_player представлен в [листинге 6](#).

```
1 /// @brief Если у игрока нет подходящих карт, он тянет дополнительную
2 /// карту и говорит, хочет ли он ее сразу положить или нет.
3 /// В остальных ситуациях кладём эту карту в руку.
4 /// @param additionalCard дополнительная карта.
5 /// @return true, если игрок сразу же кладет эту карту, иначе false.
6 bool Poly_player::drawAdditionalCard(const Card* additionalCard)
7 {
8     if (WhatCard(additionalCard) == 1 && (CheckColor(additionalCard) ||
9         CheckValue(additionalCard)))
10     {
11         return true;
12     }
13     if (WhatCard(additionalCard) == -1)
14     {
15         return !ColorAvailability();
16     }
17     if (WhatCard(additionalCard) == 0)
18     {
19         return true;
20     }
21     hand_card.push_back(additionalCard);
22     return false;
23 }
```

5 Результаты

Метод «runGames» , реализованный в исходном коде[1] в классе StatsObserver, позволяет получить Win rate игроков в заданном количестве партий. В [таблице 1](#) представлена статистика по результатам 6 игр(запусков тестовой функции «statisticTest») на 1000 партий с различным значением поля cards, описанном в разделе [4.1](#). По результатам статистики видно, что параметр cards не оказывает особого влияния на процент побед умного игрока Poly_player.

Таблица 1. Статистика по играм между Poly_player и Luser_player

Параметр cards	Количество партий	Процент выигрыша Poly_player
5	1000	94.9%
5	1000	94.0%
5	1000	94.4%
3	1000	95.6%
3	1000	96.2%
3	1000	96.4%

Заключение

В результате работы поставленные задачи были выполнены. Реализованный игрок одерживает победу практически в 95% случаев. Такой высокий результат доказывает корректность работы игрока, а именно реализованных методов в классе.

В ходе выполнения работы были изучены некоторые классы стандартной библиотеки шаблонов STL, которые также были применены на практике. Всего на курсовую работу было потрачено около 7 часов работы. Код для игроков `Poly_player` и `Luser_player` состоит из более 500 строк.

Список литературы

- [1] Описание и требования к курсовой работе // Санкт-Петербургский политехнический университет. Институт компьютерных наук и технологий. Высшая школа искусственного интеллекта [Электронный ресурс]. Режим доступа: <http://13633-oop.mooo.com/files/readme.html> (дата обращения 8.05.2023)

Приложение А. Исходный код Luser_player

А.1 Luser_player.h

```
1 #pragma once
2 #include "game/uno\_game.h"
3 using std::string;
4 using std::vector;
5 class Luser\_player : public UnoPlayer
6 {
7 private:
8     string Player\_name;
9     vector<const Card*> hand\_card;
10 public:
11     Luser\_player(string const name);
12     ~Luser\_player();
13     string name() const;
14     void receiveCards(const vector<const Card*>& cards);
15     const Card* playCard();
16     bool drawAdditionalCard(const Card* additionalCard);
17     CardColor changeColor();
18
19     //функция-проверка, какая именно карта("дикая", "дикая +4", "карта д
        ействия/обычная"
20     int WhatCard(const Card* check);
21
22     //функция-проверка на совпадение цвета определенной карты игрока с ц
        ветом карты сброса
23     bool CheckColor(const Card* color);
24
25     //функция-проверка на совпадение числа определенной карты игрока с ч
        ислom карты сброса
26     bool CheckValue(const Card* Value);
27
28     //функция удаляет карту из руки игрока после сброса
29     void Reset(const Card* last);
30
31     //функция-ситуация, когда у активного игрока нет диких карт
32     const Card* PlayerHasNoWildCards();
33
34     //функция-ситуация, когда у активного игрока есть дикие карты
35     const Card* PlayerHasWildCards();
36 };
```


A.2 Luser_player.cpp

```
1 #include "Luser\_player.h"
2 Luser\_player::Luser\_player(string const name)
3 {
4     Player\_name = name;
5 }
6
7 Luser\_player::~~Luser\_player()
8 {
9 }
10
11 string Luser\_player::name() const
12 {
13     return Player\_name;
14 }
15
16 int Luser\_player::WhatCard(const Card* check)
17 {
18     if (check->value == Wild) { return 0; }
19     if (check->value == WildDraw4) { return -1; }
20     return 1;
21 }
22
23
24 bool Luser\_player::CheckColor(const Card* Color)
25 {
26     return (Color->color == this->game()->currentColor());
27 }
28
29
30 bool Luser\_player::CheckValue(const Card* Value)
31 {
32     return (Value->value == this->game()->topCard()->value);
33 }
34
35 void Luser\_player::Reset(const Card* last)
36 {
37     int i = 0;
38     for (const Card* cur\_card : this->hand\_card)
39     {
40         if (cur\_card->color == last->color and cur\_card->value == last->
            value)
41         {
42             this->hand\_card.erase(this->hand\_card.cbegin() + i);
43             break;
44         }
45         i++;
46     }
47 }
48
49 const Card* Luser\_player:: PlayerHasNoWildCards()
50 {
51     const Card* card = nullptr;
52     for (const Card* cur\_card : this->hand\_card)
53     {
```

```

54         if (WhatCard(cur\_card) == 1 && (CheckColor(cur\_card) ||
55             CheckValue(cur\_card)))
56         {
57             card = cur\_card;
58             break;
59         }
60     return card;
61 }
62
63 const Card* Luser\_player::PlayerHasWildCards()
64 {
65     const Card* card = nullptr;
66     for (const Card* cur\_card : this->hand\_card)
67     {
68         if (WhatCard(cur\_card) == -1)
69         {
70             card = cur\_card;
71             break;
72         }
73         else if (WhatCard(cur\_card) == 0)
74         {
75             card = cur\_card;
76             break;
77         }
78     }
79     return card;
80 }
81 const Card* Luser\_player::playCard()
82 {
83     const Card* card = nullptr;
84     card = PlayerHasNoWildCards();
85     if (card != nullptr)
86     {
87         Reset(card);
88         return card;
89     }
90
91     card = PlayerHasWildCards();
92     if (card != nullptr)
93     {
94         Reset(card);
95         return card;
96     }
97 }
98
99 void Luser\_player::receiveCards(const vector<const Card*>& cards)
100 {
101     if (cards.size() == 7) { hand\_card.clear(); }
102     for (const Card* cur\_card : cards)
103     {
104         this->hand\_card.push\_back(cur\_card);
105     }
106 }
107
108 bool Luser\_player::drawAdditionalCard(const Card* additionalCard)

```

```

109 {
110     if (WhatCard(additionalCard) == 1 && (CheckColor(additionalCard) ||
111         CheckValue(additionalCard)))
112     {
113         return true;
114     }
115     if (WhatCard(additionalCard) == -1)
116     {
117         for (const Card* cur\_card : this->hand\_card)
118         {
119             if (CheckColor(cur\_card))
120             {
121                 break;
122             }
123             else { return true; };
124         }
125         if (WhatCard(additionalCard) == 0)
126         {
127             return true;
128         }
129
130         hand\_card.push\_back(additionalCard);
131         return false;
132     }
133
134 CardColor Luser\_player::changeColor()
135 {
136     srand(time(NULL));
137     int max = 3;
138     int num = rand() % (max + 1);
139
140     return CardColor(num);
141 }

```

Приложение Б. Исходный код Poly_player

Б.1 Poly_player.h

```
1 #pragma once
2 #include "../game/uno\_game.h"
3 using std::string;
4 using std::vector;
5 using std::set;
6
7 class Poly\_player : public UnoPlayer
8 {
9     string Player\_name;
10
11     vector<const Card*> hand\_card;
12
13     int cards;
14
15     // void SetCards(int num);
16
17     int GetCards();
18 public:
19     Poly\_player(string const name);
20
21     void SetCards(int num);
22
23     ~Poly\_player();
24
25     string name() const;
26
27     void receiveCards(const vector<const Card*>& cards);
28
29     const Card* playCard();
30
31     bool drawAdditionalCard(const Card* additionalCard);
32
33     CardColor changeColor();
34
35     //функция возвращает максимальное число определенного цвета
36     int MaxNumberOfCertainColor(const CardColor&);
37
38     //функция возвращает максимальное число конкретного множества(которое
39     //передали в параметрах)
40     int MaxNumber(const set<int>& set);
41
42     //функция-проверка на наличие цвета карты сброса в руке игрока
43     bool ColorAvailability();
44
45     //функция-проверка, какая именно карта("дикая", "дикая +4", "карта де
46     //йствия/обычная")
47     int WhatCard(const Card* check);
48
49     //функция-проверка на совпадение цвета определенной карты игрока с ц
50     //ветом карты сброса
51     bool CheckColor(const Card* color);
```

```

49
50 //функция-проверка на совпадение числа определенной карты игрока с ч
    ислем карты сброса
51 bool CheckValue(const Card* Value);
52
53 //функция удаляет карту из руки игрока после сброса
54 void Reset(const Card* last);
55
56 //функция возвращает количество карт соседа(направление учитывается)
57 int NumberCardNeighboringPlayer();
58
59 //функция сообщает, приближается ли соседний игрок к "УН0!"
60 bool ToxicFunction();
61
62 //функция проверяет наличие карт действия у игрока
63 bool PresenceOfActionCard();
64
65 //функция проверяет наличие диких карт у игрока
66 bool PresenceOfAWildCard();
67
68 //функция-ситуация, когда у соседнего игрока мало карт на руках, т.е
    . он приближается к "УН0!"
69 const Card* PlayerHasFewCards();
70
71 //функция-ситуация, когда у активного игрока нет диких карт
72 const Card* PlayerHasNoWildCards();
73
74 //функция-ситуация, когда у активного игрока есть дикие карты
75 const Card* PlayerHasWildCards();
76 };

```

B.2 Poly_player.cpp

```
1 #include "Poly\_player.h"
2
3 Poly\_player::Poly\_player(string const name)
4 {
5     Player\_name = name;
6     cards = 5;
7 }
8
9 Poly\_player::~Poly\_player()
10 {
11 }
12
13 string Poly\_player::name() const
14 {
15     return Player\_name;
16 }
17
18 void Poly\_player::SetCards(int num)
19 {
20     cards = num;
21 }
22
23 int Poly\_player::GetCards()
24 {
25     return cards;
26 }
27
28 int Poly\_player::NumberCardNeighboringPlayer()
29 {
30     int cards = 0;
31     int index = this->game()->activePlayerIndex();
32     std::vector<int> playersCards= game()->numberOfCards();
33     int offset = game()->currentDirection() == 0 ? 1 : -1;
34     if ((index + offset) == -1)
35     {
36         cards = playersCards[playersCards.size()-1];
37     }
38     else if ((index + offset) == playersCards.size())
39     {
40         cards = *playersCards.begin();
41     }
42     else
43     {
44         cards = playersCards[index+offset];
45     }
46     return cards;
47 }
48
49 bool Poly\_player::ToxicFunction()
50 {
51     int score = NumberCardNeighboringPlayer();
52     if (score<=cards)
53     {
54         return true;
```

```

55     }
56     return false;
57 }
58
59 int Poly\_player::WhatCard(const Card* check)
60 {
61     if (check->value == Wild) { return 0; }
62     if (check->value == WildDraw4) { return -1; }
63     return 1;
64 }
65 bool Poly\_player::PresenceOfActionCard()
66 {
67     bool flag = false;
68     for (const Card* card : hand\_card)
69     {
70         if (card->is\_action())
71         {
72             flag = true;
73         }
74     }
75     return flag;
76 }
77
78 bool Poly\_player::CheckColor(const Card* Color)
79 {
80     return (Color->color == this->game()->currentColor());
81 }
82
83
84 bool Poly\_player::CheckValue(const Card* Value)
85 {
86     return (Value->value == this->game()->topCard()->value);
87 }
88
89 bool Poly\_player::PresenceOfAWildCard()
90 {
91     bool flag = false;
92     for (const Card* card : hand\_card)
93     {
94         if (card->is\_wild())
95         {
96             flag=true;
97         }
98     }
99     return flag;
100 }
101
102 void Poly\_player:: Reset(const Card* last)
103 {
104     int i = 0;
105     for (const Card* cur\_card : this->hand\_card)
106     {
107         if (cur\_card->color == last->color and cur\_card->value == last->
            value)
108         {
109             this->hand\_card.erase(this->hand\_card.cbegin() + i);

```

```

110         break;
111     }
112     i++;
113 }
114 }
115 int Poly\_player::MaxNumber(const set<int>& set)
116 {
117     int num\_color = 0;
118     for (int i : set)
119     {
120         if (i > num\_color)
121         {
122             num\_color = i;
123         }
124     }
125     return num\_color;
126 }
127 int Poly\_player::MaxNumberOfCertainColor (const CardColor& color)
128 {
129     set<int> green;
130     set<int> red;
131     set<int> blue;
132     set<int> yellow;
133     for (const Card* card : hand\_card)
134     {
135         if (card->color == 0 && WhatCard(card) == 1)
136         {
137             red.insert(card->value);
138         }
139         if (card->color == 1 && WhatCard(card) == 1)
140         {
141             green.insert(card->value);
142         }
143         if (card->color == 2 && WhatCard(card)==1)
144         {
145             blue.insert(card->value);
146         }
147         if (card->color == 3 && WhatCard(card) ==1)
148         {
149             yellow.insert(card->value);
150         }
151     }
152     if (color == 0)
153     {
154         return MaxNumber(red);
155     }
156     if (color == 1)
157     {
158         return MaxNumber(green);
159     }
160     if (color == 2)
161     {
162         return MaxNumber(blue);
163     }
164     if (color == 3)
165     {

```



```

166         return MaxNumber(yellow);
167     }
168 }
169 bool Poly\_player::ColorAvailability()
170 {
171     bool flag = false;
172     for (const Card* cur\_card : hand\_card)
173     {
174         if (cur\_card->color == this->game()->currentColor() && !cur\_card
            ->is\_wild())
175         {
176             flag = true;
177         }
178     }
179     return flag;
180 }
181
182 const Card* Poly\_player:: PlayerHasFewCards()
183 {
184     const Card* card = nullptr;
185     if (ToxicFunction() && (PresenceOfAWildCard() || PresenceOfActionCard
        ()))
186     {
187         for (const Card* cur\_card : hand\_card)
188         {
189             if (cur\_card->value == WildDraw4 && !ColorAvailability())
190             {
191                 card = cur\_card;
192                 break;
193             }
194             else if (cur\_card->value == Wild)
195             {
196                 card = cur\_card;
197                 break;
198             }
199             else if (cur\_card->is\_action() && CheckColor(cur\_card))
200             {
201                 card = cur\_card;
202                 break;
203             }
204         }
205     }
206     return card;
207 }
208
209 const Card* Poly\_player::PlayerHasNoWildCards()
210 {
211     const Card* card = nullptr;
212     CardColor cur\_color = game()->currentColor();
213     for (const Card* cur\_card : hand\_card)
214     {
215         if (WhatCard(cur\_card) == 1 && CheckValue(cur\_card))
216         {
217             card = cur\_card;
218             break;
219         }

```

```

220         else if (WhatCard(cur\_card) == 1
221                 && CheckColor(cur\_card)
222                 && (cur\_card->value == MaxNumberOfCertainColor(cur\_color)
223                     ))
224             {
225                 card = cur\_card;
226                 break;
227             }
228         return card;
229     }
230
231     const Card* Poly\_player::PlayerHasWildCards()
232     {
233         const Card* card = nullptr;
234         for (const Card* cur\_card : hand\_card)
235         {
236             if (WhatCard(cur\_card) == -1 && !ColorAvailability())
237             {
238                 card = cur\_card;
239                 break;
240             }
241             else if (WhatCard(cur\_card) == 0)
242             {
243                 card = cur\_card;
244                 break;
245             }
246         }
247         return card;
248     }
249
250
251     const Card* Poly\_player::playCard()
252     {
253         const Card* card = nullptr;
254         card = PlayerHasFewCards();
255         if (card != nullptr)
256         {
257             Reset(card);
258             return card;
259         }
260         card = PlayerHasNoWildCards();
261         if (card != nullptr)
262         {
263             Reset(card);
264             return card;
265         }
266         card = PlayerHasWildCards();
267         if (card != nullptr)
268         {
269             Reset(card);
270             return card;
271         }
272     }
273
274     void Poly\_player::receiveCards(const vector<const Card*>& cards)

```

```

275 {
276     if (cards.size() == 7)
277     {
278         hand\_card.clear();
279     }
280     for (const Card* cur\_card : cards)
281     {
282         this->hand\_card.push\_back(cur\_card);
283     }
284 }
285
286 bool Poly\_player::drawAdditionalCard(const Card* additionalCard)
287 {
288     if (WhatCard(additionalCard) == 1 && (CheckColor(additionalCard) ||
289         CheckValue(additionalCard)))
290     {
291         return true;
292     }
293     if (WhatCard(additionalCard) == -1)
294     {
295         return !ColorAvailability();
296     }
297     if (WhatCard(additionalCard) == 0)
298     {
299         return true;
300     }
301     hand\_card.push\_back(additionalCard);
302     return false;
303 }
304
305 CardColor Poly\_player::changeColor()
306 {
307     int num\_color[4] = {0};
308     for (const Card* cur\_card : hand\_card)
309     {
310         switch (cur\_card->color)
311         {
312             case 0:
313                 num\_color[0] += 1;
314                 break;
315             case 1:
316                 num\_color[1] += 1;
317                 break;
318             case 2:
319                 num\_color[2] += 1;
320                 break;
321             case 3:
322                 num\_color[3] += 1;
323                 break;
324         }
325     }
326     int index = 0;
327     int max = 0;
328     for (int i = 0; i < 4; i++)

```

```
330     {
331         if (num\_color[i] > max)
332         {
333             max = num\_color[i];
334             index = i;
335         }
336     }
337     return CardColor(index);
338 }
```